



Факултет за информатички науки и компјутерско инженерство

Проектна задача

Предмет:
Тимски проект

Тема:
ApexInsight - F1 Race Prediction Platform

Изработиле:

Илија Бунчески – 221094
Марија Димова – 221288
Софија Речкоска – 221064
Филип Иваноски – 221038

Ментор:

Проф. Д-р Андреа Кулаков

Скопје
Јуни, 2026



Содржина

1. Вовед.....	3
2. Цел и опфат на проектот	3
3. Архитектура на системот	4
4. Технологии и алатки	6
Backend.....	6
База на податоци	6
Data и ML	6
Frontend	6
Containers	6
5. Модел на податоци.....	6
Главни табели	7
6. Прибирање и обработка на податоци	8
7. Машинско учење и предвидувања	9
Карактеристики што се користат	9
Модели	10
8. Backend API.....	11
Главни API групи	12
Error handling и request tracking.....	12
9. Frontend апликација.....	12
10. Главни функционалности	13
Сезонски преглед	13
Race selector	13
Анализа на трка	13
Стратегија со гуми	13
Споредба на возачи	14
Предвидување за трка	14
Предвидување за следна трка.....	14
Споредба на предвидувања	14
11. Генерирање и користење предвидувања	17
12. Инсталација и стартување.....	19
13. Верификација и тестирање	20
14. Ограничувања	21
15. Идни подобрувања.....	21
16. Заклучок	22

1. Вовед

Проектот **F1 Race Prediction Platform** претставува full-stack веб апликација за анализа на Formula 1 податоци и генерирање предвидувања за резултати од трки. Апликацијата комбинира повеќе делови: прибирање историски податоци, складирање во релациона база, обработка на карактеристики за машинско учење, тренирање модели и интерактивен frontend dashboard преку кој корисникот може да ги разгледува резултатите.

Основната идеја на проектот е да се претстават Formula 1 податоците на начин што е корисен и за анализа и за предвидување. Наместо апликацијата да биде само листа со трки и резултати, таа нуди неколку аналитички погледи: сезонски преглед, анализа на конкретна трка, споредба на возачи, стратегија со гуми, споредба на предвидувања со реални резултати и објаснување на ML моделите.

Formula 1 е погоден домен за ваков проект затоа што содржи богати и временски зависни податоци. Резултатите не зависат само од еден фактор, туку од комбинација на квалификациска позиција, форма на возачот, форма на тимот, историја на одредена патека, темпо во претходни трки, временски услови, стратегија со гуми и други непредвидливи фактори. Поради тоа, проектот е добар пример за интеграција на backend систем, база, ML pipeline и frontend визуелизација.

Во рамки на оваа документација се објаснува што е имплементирано во проектот, кои технологии се користат, како тече обработката на податоците, како се генерираат предвидувањата и кои функционалности се достапни во корисничкиот интерфејс.

2. Цел и опфат на проектот

Главната цел на проектот е да се изгради платформа што овозможува:

- преглед на Formula 1 сезони, трки, возачи и тимови;
- анализа на резултати од трки и кругови;
- визуелизација на темпо, промени на позиции, најбрзи кругови и временски услови;
- анализа на стратегија со гуми;
- споредба на перформанси помеѓу двајца возачи;
- генерирање ML предвидувања за исход од трка;
- споредба помеѓу предвидени и реални резултати;
- објаснување на моделите, метриците и важноста на карактеристиките.

Проектот не е замислен како betting систем или како алатка што гарантира точен исход на трка. Неговата цел е аналитичка и едукативна: да покаже како може да се спојат податоци, API, база, машинско учење и интерактивен UI во една целина.

Опфатот на проектот ги вклучува следните компоненти:

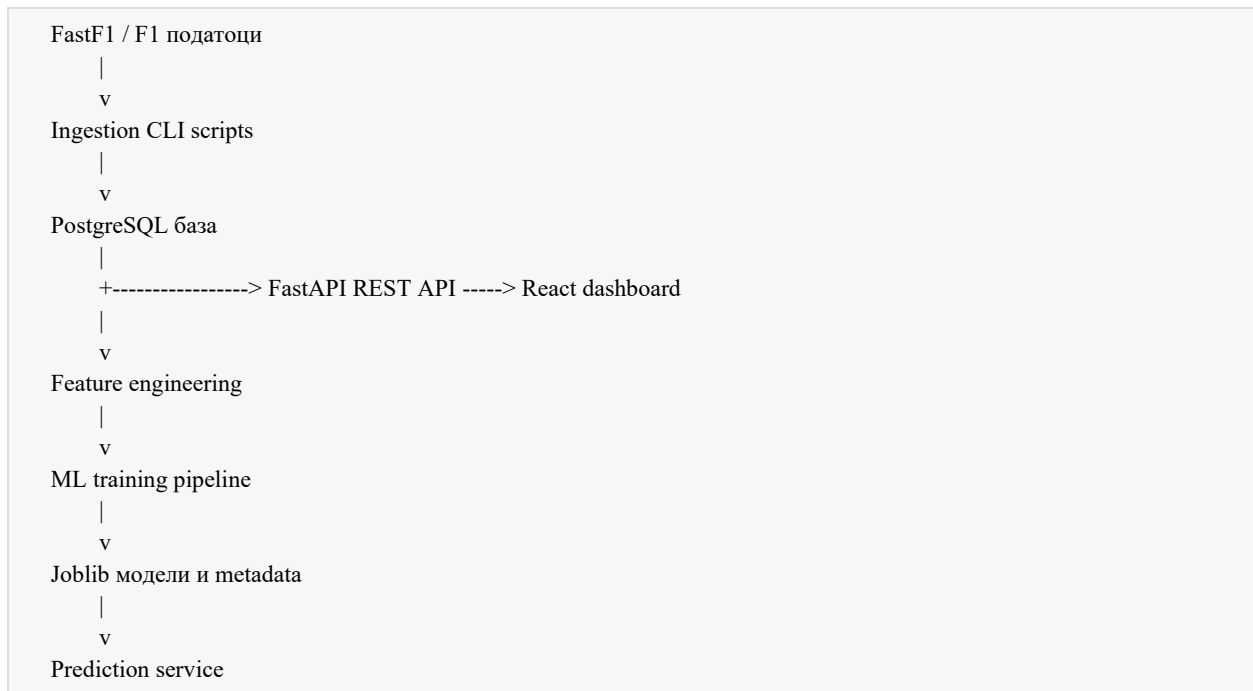
- **Backend апликација** изградена со FastAPI, која обезбедува REST API за frontend апликацијата.

- **PostgreSQL база** во која се чуваат нормализирани податоци за сезони, трки, возачи, тимови, резултати, кругови, временски услови, ML features и предвидувања.
- **Ingestion scripts** кои преземаат и внесуваат податоци од Formula 1 изворите преку FastF1.
- **ML pipeline** за feature engineering, тренирање модели и зачувување на модели во [joblib](#) формат.
- **React/Vite frontend** со TypeScript, React Query, Recharts и Tailwind CSS.
- **Docker Compose околина** за стартување на базата, backend сервисот, frontend сервисот и ingestion алатките.

Проектот е организиран така што секој слој има јасна одговорност. Базата го чува доменскиот модел, backend API-то го контролира пристапот до податоците и бизнис логиката, ML pipeline-от ги подготвува и тренира моделите, а frontend-от ги претставува податоците на корисникот преку визуелни компоненти.

3. Архитектура на системот

Системот е изграден како класична full-stack архитектура со издвоени сервиси. Главниот тек на податоци започнува од историските Formula 1 податоци, продолжува низ ingestion скриптите, се складира во PostgreSQL, потоа се обработува преку feature engineering, се користи за тренирање ML модели и на крај се презентира преку API и React dashboard.

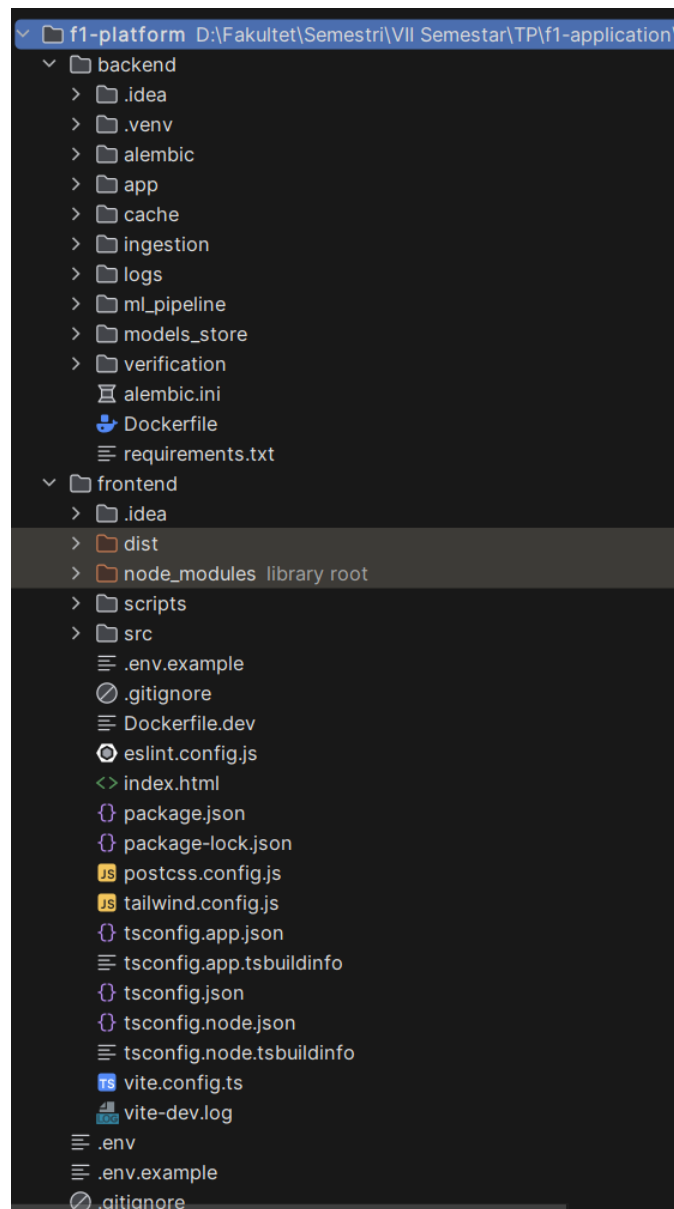


Во Docker Compose конфигурацијата се дефинирани четири главни сервиси:

- **db:** PostgreSQL 16 база на податоци.
- **backend:** FastAPI апликација што работи на портата 8000.

- **frontend**: Vite React апликација што работи на портата 5173.
- **ingestion**: алатка активирана по потреба преку Docker profile, наменета за внесување и обработка на податоци.

Backend сервисот при стартување ја проверува конекцијата со базата, ги вчитува ML моделите од директориумот за модели и го изложува API-то. Frontend сервисот комуницира со backend-от преку HTTP повици и ги кешира податоците со React Query. Ingestion сервисот не работи постојано, туку се стартува кога треба да се внесат нови сезони, резултати, кругови, временски податоци или ML features.



Слика 1. Проектна структура

4. Технологии и алатки

Проектот користи комбинација на backend, frontend, database и machine learning технологии.

Backend

Backend-от е изграден со **FastAPI**, Python framework за брза изработка на REST API. FastAPI овозможува автоматски OpenAPI schema, Swagger документација, dependency injection и валидација на податоци. За ORM слојот се користи **SQLAlchemy 2.0**, а за миграции се користи **Alembic**. Конфигурацијата се управува преку environment променливи и Pydantic settings.

Backend апликацијата има и middleware за **X-Request-ID**, централизирано враќање на грешки и production API key проверка. Ова значи дека системот не е само функционален, туку има и основни механизми за следење, дебагирање и контрола на пристап.

База на податоци

За складирање се користи **PostgreSQL 16**. Податоците се нормализирани во повеќе табели, како seasons, races, drivers, teams, race_results, qualifying_results, lap_times, weather_data, ml_features и predictions. Дел од табелите имаат unique constraints и индекси за побрзо пребарување и заштита од дупликати.

Data и ML

За прибирање на Formula 1 податоци се користи **FastF1**. За обработка и анализа се користат **pandas**, **NumPy**, **scikit-learn**, **XGBoost**, **LightGBM** и **joblib**. Моделите се тренираат како pipeline-и што вклучуваат imputation, scaling и на крај применување на ML алгоритми.

Frontend

Frontend-от е изграден со **React**, **TypeScript** и **Vite**. За податочни повици и кеширање се користи **TanStack React Query**, за графикони **Recharts**, за рутирање **React Router**, а за изгледот на апликацијата **Tailwind CSS**. UI-то е организирано во страници и помали реискористливи компоненти.

Containers

Целата околина се стартува преку **Docker Compose**. Ова го поедноставува локалното стартување затоа што корисникот не мора рачно да поставува база, backend и frontend сервиси одделно.

5. Модел на податоци

Базата е поставена околу Formula 1 доменот. Главните ентитети се сезони, трки, возачи, тимови и резултати. Околу нив се додаваат детални податоци за квалификации, кругови, временски услови, ML features и предвидувања.



Главни табели

seasons ја претставува една F1 сезона. Содржи година, број на трки, шампион возач и шампион тим. Оваа табела е основа за групирање на трките.

races ја претставува секоја трка во рамки на сезона. Содржи реден број, име на трка, име и локација на патеката, држава, датум и session key. Има unique constraint за комбинацијата сезона и round number, со што се спречува дуплирање на иста трка во сезона.

drivers ги чува возачите, нивните броеви, кратенки, националност и поврзаност со тим. **teams** ги чува конструкторите, кратките имиња и националностите.

race_results ги чува конечните резултати од трката: стартна позиција, завршна позиција, статус, поени, завршени кругови, најбрз круг и најбрзо време. Табелата има уникатна комбинација **race_id** и **driver_id**, бидејќи еден возач треба да има само еден резултат по трка.

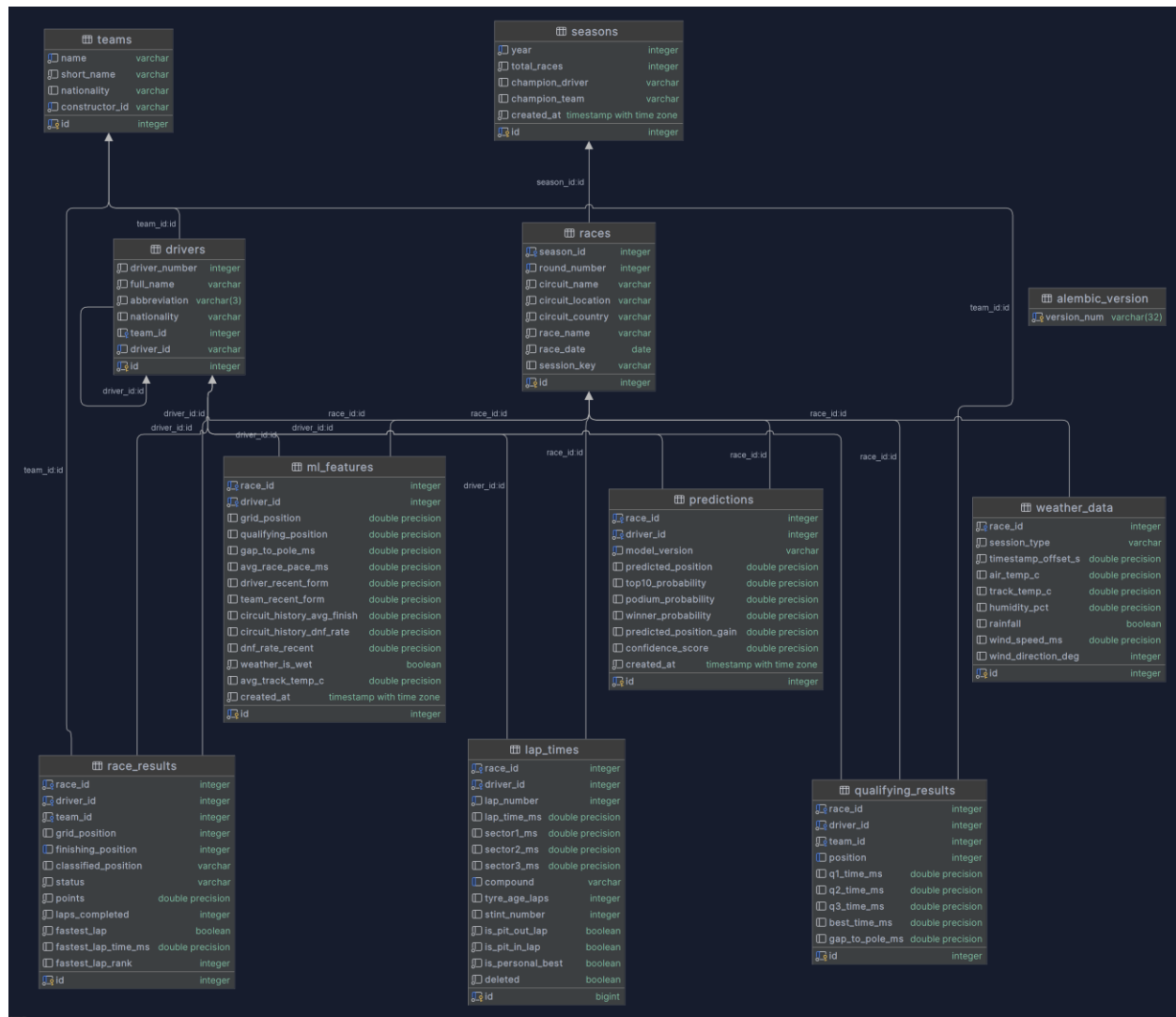
qualifying_results ги чува резултатите од квалификации, вклучувајќи Q1, Q2, Q3 времиња, најдобро време и разлика до pole position.

lap_times ги чува круговите на возачите. Освен времето на круг, се чуваат и секторски времиња, compound на гумата, старост на гумата, stint број и информации дали кругот е pit in/out или личен најдобар круг.

weather_data ги чува временските услови по сесија: температура на воздух, температура на патека, влажност, дожд, брзина и насока на ветер.

ml_features е табела за подготвени карактеристики за машинско учење. Таа содржи вредности како стартна позиција, квалификациска позиција, просечно темпо, форма на возач, форма на тим, историја на патеката, DNF стапка и временски услови.

predictions ги чува резултатите од ML предвидувањата: предвидена позиција, веројатност за top 10, веројатност за podium, веројатност за победа, предвидена промена на позиција и confidence score.



Слика 2. Дијаграм од табелите во базата на податоци

6. Прибирање и обработка на податоци

Податоците се внесуваат преку ingestion скрипти во backend делот. Скриптите се поделени според типот на податок што го обработуваат:

- `ingest_core.py`: внесување основни информации за сезони, трки, возачи и тимови;
- `ingest_results.py`: внесување резултати од трки и квалификации;
- `ingest_laps.py`: внесување кругови, секторски времиња, гуми и временски услови;
- `ingest_seasons.py`, `ingest_races.py`, `ingest_weekend.py`: помошни ingestion скрипти за специфични делови од процесот.

Процесот може да се изврши преку Docker Compose. Пример за внесување на основни податоци:


```
docker-compose run --rm ingestion python ingestion/ingest_core.py --seasons 2021 2022 2023 2024
```

Потоа се внесуваат резултати:

```
docker-compose run --rm ingestion python ingestion/ingest_results.py --seasons 2021 2022 2023 2024
```

На крај се внесуваат кругови и временски услови:

```
docker-compose run --rm ingestion python ingestion/ingest_laps.py --seasons 2021 2022 2023 2024
```

По внесувањето, се извршува feature engineering:

```
docker-compose run --rm ingestion python ml_pipeline/feature_engineering.py --seasons 2021 2022 2023 2024
```

Feature engineering делот има важна улога затоа што суровите податоци не се секогаш директно погодни за ML модел. На пример, за да се предвиди резултат на трка, не е доволно да се знае само името на возачот. Потребни се агрегирани и контекстуални карактеристики: просечно темпо од претходни трки, форма на возачот, форма на тимот, историја на патеката, веројатност за DNF, температура на патеката и слично.

Во проектот се поддржани два feature контексти:

- **pre_qualifying**: предвидување пред квалификации, кога не се знае стартната позиција;
- **post_qualifying**: предвидување по квалификации, кога се знаат квалификациската позиција, стартната позиција и гар до pole.

Ова е значајно затоа што апликацијата може да работи со различно ниво на достапни информации. Пред квалификации, моделот се потпира повеќе на историска форма и контекст. По квалификации, моделот добива дополнителни сигнали што најчесто се многу важни за конечниот резултат.

7. Машинско учење и предвидувања

ML делот на проектот е организиран во два главни чекори: подготовка на feature rows и тренирање модели. Feature rows се чуваат во табелата `ml_features`, а тренираните модели се зачувуваат во директориумот `models_store`.

Карактеристики што се користат

За **pre_qualifying** контекстот се користат карактеристики како:

- просечно тркачко темпо во милисекунди;
- форма на возачот во последните трки;
- форма на тимот;
- просечен резултат на истата патека;
- DNF стапка на патеката и во последните трки;
- дали условите се влажни;
- просечна температура на патеката.

За **post_qualifying** контекстот се додаваат и:

- стартна позиција;
- квалификациска позиција;
- разлика до pole position во милисекунди.

Модели

Проектот тренира повеќе модели, бидејќи еден единствен модел не е доволен за сите типови предвидување:

- `position_model`: регресиски модел за предвидување на завршна позиција;
- `top10_model`: класификациски модел за веројатност возачот да заврши во top 10;
- `podium_model`: класификациски модел за веројатност возачот да заврши на podium;
- `position_gain_model`: регресиски модел за очекувана добивка или загуба на позиции.

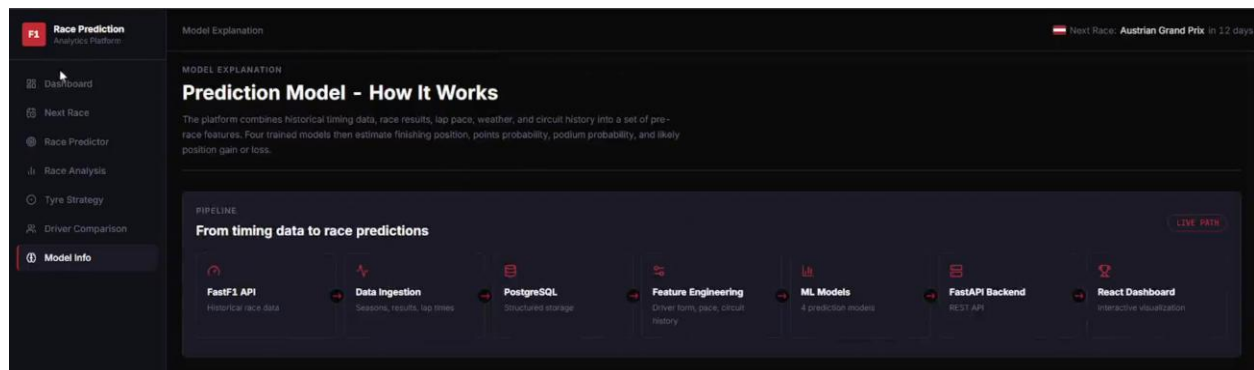
За регресија се споредуваат алгоритми како `RandomForestRegressor`, `XGBRegressor` и `LGBMRegressor`, а се избира најдобриот според MAE. За top 10 и podium се користи `XGBClassifier`. Моделите се тренираат со временска поделба: сезони за тренинг и посебна сезона за тестирање. Ова е важно затоа што Formula 1 податоците се временски зависни и не треба идни информации да се користат за предвидување на минатото.

Пример команда за тренирање:

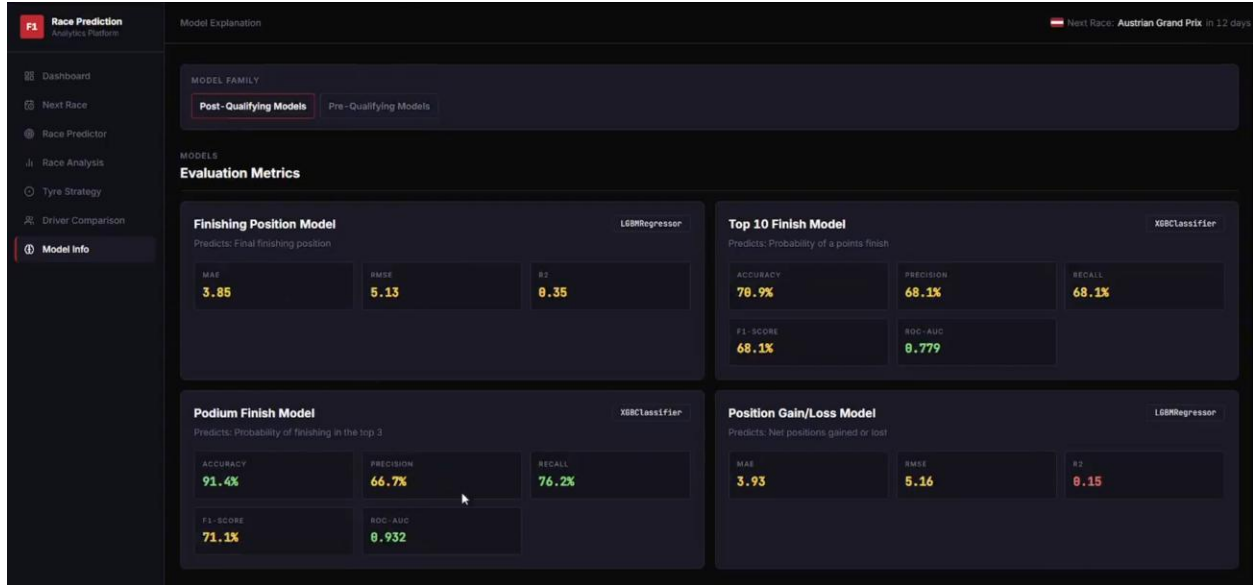
```
docker-compose run --rm ingestion python ml_pipeline/train_models.py --train-seasons 2021 2022 2023 --test-season 2024 --context all
```

По тренирањето се зачувуваат:

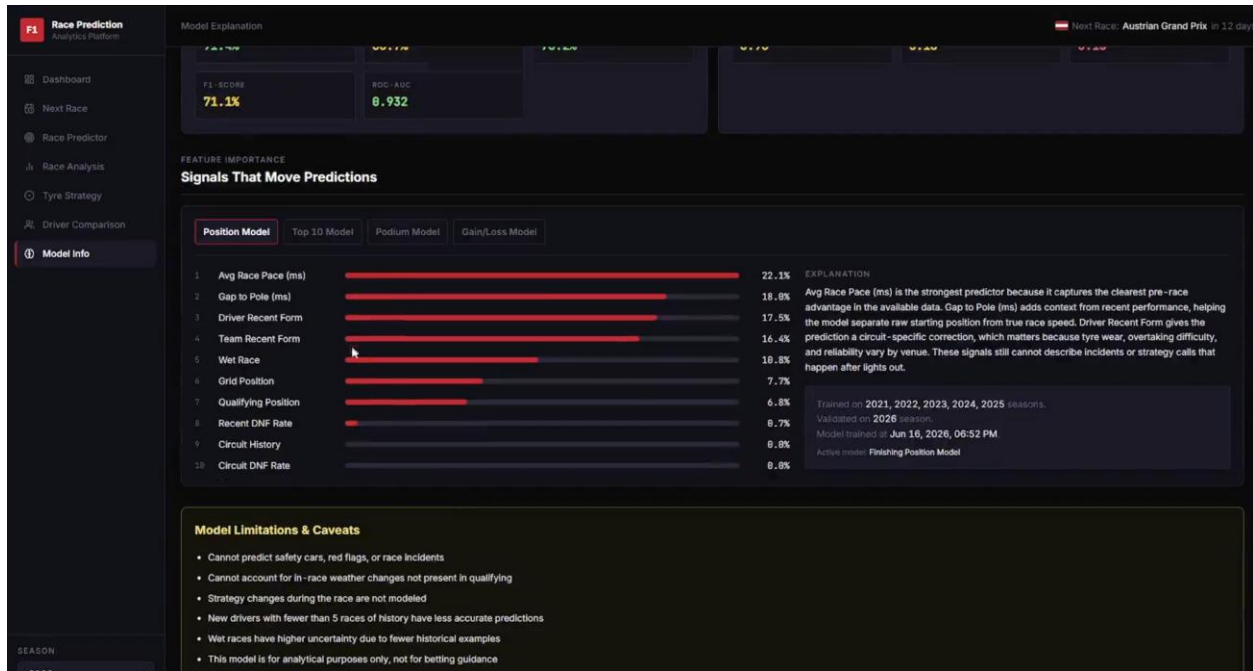
- `.joblib` датотеки со модели;
- JSON metadata за моделите;
- JSON датотеки со feature importances;
- evaluation report текстуални датотеки.



Слика 3. Pipeline – Data Flow



Слика 4. Evaluation Metrics



Слика 5. Feature Importance за секој модел

8. Backend API

Backend API-то е имплементирано со FastAPI и е достапно преку префиксот `/api/v1`. Дополнително, автоматската API документација е достапна на:

```
http://localhost:8000/api/docs
```

Health check endpoint-от е:

```
http://localhost:8000/health
```

Тој враќа статус на апликацијата, околината, базата и дали ML моделите се вчитани.

Главни API групи

[/seasons](#) овозможува листа на сезони, детали за конкретна сезона и статистики за сезона.

[/races](#) овозможува листа на трки, детали за трка, следна трка, квалификациски резултати и резултати од трка.

[/drivers](#) овозможува листа на возачи, детали за возач и статистики за возач.

[/teams](#) овозможува листа на тимови, детали за тим и статистики за тим.

[/analysis](#) ги опфаќа аналитичките endpoints за кругови, lap summary, стратегија со гуми, промени на позиции, споредба на возачи, најбрзи кругови и временски услови.

[/predictions](#) ги опфаќа endpoint-ите за генерирање предвидувања, приказ на предвидувања за трка, предвидување за следна трка, споредба на предвидувања со реални резултати, feature importances и информации за моделите.

Error handling и request tracking

Backend-от има централизирано справување со грешки. Кога ќе се случи грешка, API-то враќа JSON одговор со:

- `error`;
- `message`;
- `request_id`;
- `timestamp`.

`request_id` се додава и во response header како `X-Request-ID`. Ова помага при дебагирање, бидејќи е полесно да се поврзе кориснички проблем со backend логови.

Во production околина, API-то поддржува API key заштита преку `X-API-Key` header за `/api/v1` патеките.

9. Frontend апликација

Frontend-от е React апликација изградена со TypeScript и Vite. Главната рута е организирана во `App.tsx`, а страниците се сместени во `src/pages`. Апликацијата користи `AppShell`, кој обезбедува заедничка структура со sidebar, top bar и простор за содржина.

Главните frontend рути се:

- `/seasons/:year` - преглед на сезона;
- `/seasons/:year/races` - избор на трка;

- [/seasons/:year/races/:raceId/predict](#) - предвидување за конкретна трка;
- [/predictions/next-race](#) - предвидување за следна трка;
- [/seasons/:year/races/:raceId/prediction-comparison](#) - споредба на предвидувања со реални резултати;
- [/seasons/:year/races/:raceId/analysis](#) - анализа на трка;
- [/seasons/:year/races/:raceId/tyres](#) - стратегија со гуми;
- [/drivers/compare](#) - споредба на возачи;
- [/model](#) - објаснување на моделот.

Податоците се добиваат преку API wrapper-и во [src/api](#), а типови се дефинирани во [src/types](#). React Query се користи за кеширање, loading states и повторно користење на API резултати. UI компонентите како [StatCard](#), [LoadingSpinner](#), [ErrorState](#), [EmptyState](#), [TeamLogo](#), [CountryFlag](#), [PositionBadge](#) и [TyreBadge](#) помагаат интерфејсот да биде конзистентен низ повеќе страници.

Визуелизациите се прават со [Recharts](#). Така, корисникот не гледа само табели, туку и графикони за промена на позиции, темпо по круг, распределба на гасе расе, секторска споредба, грешка на предвидување и важност на features.

10. Главни функционалности

Сезонски преглед

Страницата за сезонски преглед прикажува основни информации за избраната сезона, статистики и табели со standings. Корисникот може да ја види структурата на сезоната и да продолжи кон календарот со трки. Ова е почетната точка од која се навигира кон подетални анализи.

Race selector

Race selector страницата овозможува избор на трка во рамки на сезона. Таа служи како мост помеѓу сезонскиот приказ и деталните страници за анализа, предвидување, гуми и споредба.

Анализа на трка

Страницата [Race Analysis](#) ги комбинира најважните податоци за конкретна трка. Во неа се прикажуваат резултати, најбрзи кругови, временски услови, графикон на промени на позиции и графикон на lap times. Ова му дава на корисникот целосна слика за тоа како се развивала трката.

Стратегија со гуми

Страницата [Tyre Strategy](#) прикажува податоци за compounds, stint-ови и pit stops. Таа е важна затоа што стратегијата со гуми има големо влијание врз Formula 1 резултатите. Корисникот може да види кој возач кога користел одреден compound и како тоа се поврзува со темпото.



Споредба на возачи

Страницата [Driver Comparison](#) овозможува директна споредба на двајца возачи. Се прикажуваат head-to-head статистики, delta во lap times, распределба на race расе и секторска споредба. Ова е корисно за анализа на перформанси во рамки на иста трка.

Предвидување за трка

Страницата [Race Predictor](#) овозможува генерирање и прикажување предвидувања за избрана трка. Табелата со предвидувања содржи предвидена позиција, top 10 probability, podium probability, winner probability, очекувана промена на позиција и confidence score. Корисникот може да избере prediction context, што е важно за разликата помеѓу пред и по квалификации.

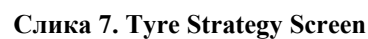
Предвидување за следна трка

Страницата [Next Race Prediction](#) е фокусирана на најблиската претстојна трка. Таа прикажува контекст за трката, достапност на податоци и табела со предвидувања. Оваа функционалност е практична затоа што корисникот не мора рачно да ја бара следната трка.

Споредба на предвидувања

Страницата [Prediction Comparison](#) се користи кога за трката веќе постојат реални резултати. Таму се споредува што предвидел моделот со тоа што навистина се случило. Се прикажуваат outcome badges, accuracy summary, error chart и табела со разлики.





11. Генерирање и користење предвидувања

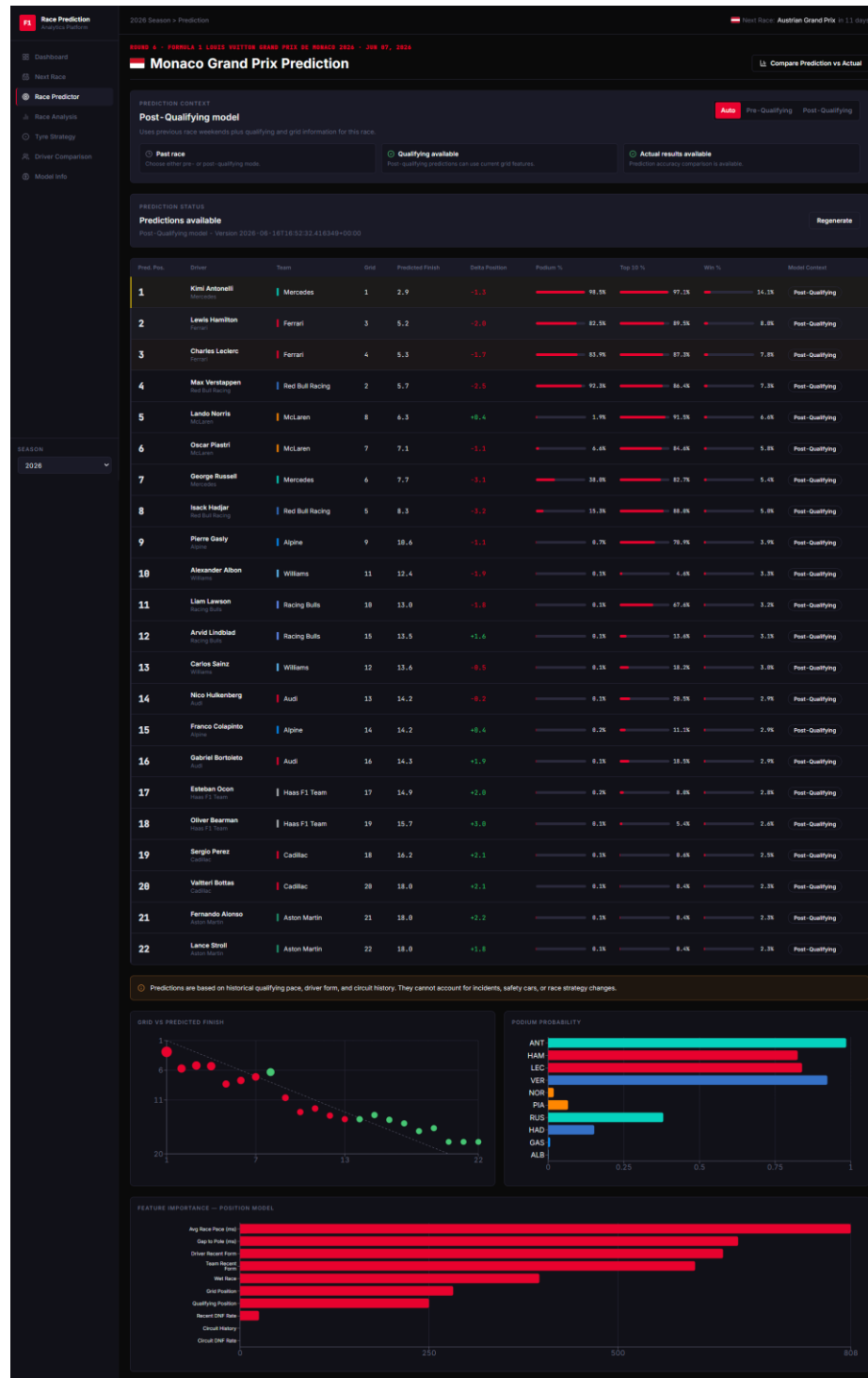
Предвидувањата во апликацијата не се статички вредности, туку се генерираат преку backend сервис што ги користи подготвените ML features и вчитаните модели. Кога корисникот бара предвидување, backend-от проверува дали постојат потребните feature rows, го избира соодветниот моделски контекст и ги запишува резултатите во табелата [predictions](#).

Еден запис за предвидување содржи:

- трка;
- возач;
- prediction context;
- model context;
- feature context;
- верзија на модел;
- предвидена позиција;
- top 10 probability;
- podium probability;
- winner probability;
- предвидена промена на позиција;
- confidence score;
- време на генерирање.

Овој дизајн овозможува повеќе предвидувања за истата трка да се разликуваат според контекст и верзија на модел. На пример, предвидување направено пред квалификации не треба да се меша со предвидување направено по квалификации, бидејќи користи различен сет на информации.

Во frontend делот, предвидувањата се претставени во табела и графикони. Корисникот може визуелно да види кои возачи имаат најголема веројатност за podium, кои се очекува да добијат позиции и каде моделот има поголема или помала сигурност.



Слика 8. Race Predictor Screen



12. Инсталација и стартување

За стартување на проектот потребни се Docker и Docker Compose. Прво се подготвува `.env` датотека според `environment template`. Потоа се стартуваат сервисите:

```
docker-compose up --build -d
```

По стартување, frontend апликацијата е достапна на:

```
http://localhost:5173
```

Backend API документацијата е достапна на:

```
http://localhost:8000/api/docs
```

Health check:

```
http://localhost:8000/health
```

Типичен редослед за подготовка на податоци е:

```
docker-compose run --rm ingestion python ingestion/ingest_core.py --seasons 2021 2022 2023 2024
docker-compose run --rm ingestion python ingestion/ingest_results.py --seasons 2021 2022 2023 2024
docker-compose run --rm ingestion python ingestion/ingest_laps.py --seasons 2021 2022 2023 2024
docker-compose run --rm ingestion python ml_pipeline/feature_engineering.py --seasons 2021 2022 2023 2024
docker-compose run --rm ingestion python ml_pipeline/train_models.py --train-seasons 2021 2022 2023 --test-season 2024
--context all
```

Корисни команди:

```
docker-compose ps
docker-compose logs -f backend
docker-compose logs -f frontend
docker-compose run --rm ingestion alembic upgrade head
```

Во frontend проектот постојат и npm scripts:

```
npm run dev
npm run build
npm run lint
npm run verify:prediction-routes
```

Овие команди се користат за локален развој, build, lint проверка и верификација на prediction routes.

13. Верификација и тестирање

Проектот содржи повеќе verification scripts во backend директориумот. Тие се наменети за проверка на prediction workflow, контексти на предвидување, споредба на предвидувања и API за следна трка. Примери се:

- `verify_prediction_workflow_e2e.py`;
- `verify_prediction_contexts.py`;
- `verify_prediction_comparison.py`;
- `verify_next_race_prediction_api.py`;
- `verify_future_races.py`.

Овие скрипти служат како практична проверка дека backend и ML делот работат заедно. Особено е важно да се провери prediction workflow-от, бидејќи тој зависи од повеќе слоеви: база, features, модели, API service и frontend повици.

Backend-от има и health check endpoint. Тој е корисен за Docker healthcheck, но и за рачна проверка дали API-то е активно и дали моделите се вчитани.

Frontend делот може да се провери преку build и lint:

```
npm run build
npm run lint
```

Дополнително, апликацијата треба рачно да се провери преку кориснички сценарија:

1. Отворање сезонски преглед.
2. Избор на трка од календар.
3. Отворање Race Analysis.
4. Отворање Tyre Strategy.
5. Споредба на двајца возачи.
6. Генерирање предвидување за трка.
7. Проверка на Next Race Prediction.
8. Споредба на предвидување со реален резултат.
9. Отворање Model Explanation.

Овој пристап комбинира автоматизирана и рачна проверка. Автоматизираната проверка потврдува дека API и workflow-от работат, додека рачната проверка потврдува дека корисничкото искуство е разбирливо и дека податоците се прикажуваат правилно.

14. Ограничувања

Иако проектот имплементира комплетен аналитички и prediction workflow, постојат природни ограничувања. Formula 1 резултатите зависат од многу непредвидливи фактори што не можат целосно да се моделираат со историски податоци.

Главни ограничувања:

- моделот не може да предвиди safety car, red flag или инциденти;
- не може целосно да се моделираат стратегиски промени за време на трката;
- временските услови можат да се променат за време на трката;
- нови возачи со малку историски податоци имаат поголема несигурност;
- влажни трки имаат поголема неизвесност поради помал број релевантни примери;
- предвидувањата се аналитички и не се наменети за обложување.

Исто така, точноста на моделите зависи од квалитетот и комплетноста на внесените податоци. Ако недостасуваат кругови, квалификации или временски податоци, feature engineering процесот мора да користи fallback вредности или медијани. Тоа е практично решение, но секогаш внесува одредено ниво на несигурност.

15. Идни подобрувања

Проектот може да се надгради во повеќе насоки:

- подобро моделирање на pit stop стратегија;
- посебен модел за DNF ризик;
- споредба на повеќе верзии на модели во UI;
- додавање user accounts и зачувани анализи;
- понапредни графикони за stint pace и degradation;
- export на анализи во PDF;
- dashboard за model monitoring.

Една важна насока е подобро објаснување на предвидувањата. Feature importance веќе дава корисна слика, но во иднина може да се додаде per-driver explanation, каде за секој возач ќе се гледа кои фактори најмногу влијаеле врз неговото предвидување.

16. Заклучок

F1 Race Prediction Platform претставува full-stack апликација која обединува повеќе важни области од современиот софтверски развој: прибирање и обработка на податоци, работа со база на податоци, backend архитектура, машинско учење и интерактивна frontend визуелизација. Проектот не се ограничува само на едноставен приказ или CRUD операции, туку обработува реални податоци од Formula 1, ги трансформира во корисни карактеристики, тренира модели за предвидување и ги прикажува резултатите на начин што е разбирлив и корисен за крајниот корисник.

Податоците се прибираат од реален извор, се складираат во база, се обработуваат преку посебен pipeline, се користат за тренирање модели и потоа се изложуваат преку backend API кон кориснички интерфејс. Ваквиот пристап ја прави апликацијата практична, проширлива и блиска до реални data-driven системи. Истовремено, проектот покажува дека предвидувањата во спорт како Formula 1 не треба да се третираат како апсолутно точни резултати, туку како проценки базирани на историски податоци и достапни показатели, бидејќи на исходот можат да влијаат непредвидливи фактори како безбедносно возило, дефекти, казни, временски промени и тимски стратегии.

Како целина, проектот успешно демонстрира како современи технологии можат да се комбинираат за изградба на функционална и аналитички ориентирана апликација. F1 Race Prediction Platform претставува пример за систем кој не само што прикажува податоци, туку ги користи за анализа, споредба и предвидување, со што се добива комплетно решение што ги поврзува софтверското инженерство и машинското учење во практичен и реален контекст.